

Project #3

TLC59208F Control

Stein Lysberg

May 1, 2026

Contents

1	Introduction	1
1.1	Configuration	1
1.1.1	Finding the I ² C address	1
1.1.2	Devicetree	1
1.1.3	Registers	2
1.2	Control	2
1.3	Testing	2
2	Discussion	3
2.1	I ² C-address	3
2.2	Control registers	3
2.3	PWM vs. LEDOUT	4
2.4	Visibility of LEDs	4
3	Conclusion	4
	References	5

1 Introduction

This report is the third in a series of five, and looks at the use of a Texas Instruments TLC59208F LED driver (TLC), for use with a Nordic Semiconductor nRF52DK (nRF52), to make a plastic scanner. The TLC will be used to control six LEDs, and make them blink in a sequence. In later assignments a photodiode will be used to read how the light from these LEDs reflects off some object, to try to determine what type of plastic it contains. This report considers configuration, control functions, and testing. It also compares this projects implementation to that of the Arduino code the project is based on.

1.1 Configuration

The TLC has a range of configuration options, so before it can be used it need to be configured for this specific project. This is done by writing to its control registers. In order to write to these registers, communication between the TLC and the nRF52 needs to be established, which is done using I²C. The Zephyr RTOS has many functions for I²C communication, and to access these, the TLC must be added to the devicetree.

ADDRESS INPUT			SLAVE ADDRESS							ADDRESS
A2	A1	A0	A6	A5	A4	A3	A2	A1	A0	
GND	GND	GND	0	1	0	0	0	0	0	40h

Figure 1: Address table for the TLC (cropped) [1].

1.1.1 Finding the I²C address

To communicate with the TLC, the devices I²C-address must be acquired. The method for determining and finding the address is found in the user manual [1]. As fig. 1 shows, when all address inputs of the TLC are grounded (as they are in the plastic scanner, shown in fig. 2), the I²C-address is 0b00100000, or 0x20, as shown by bits A6-A0.

1.1.2 Devicetree

When the I²C-address for the TLC is known, it can be added to the devicetree. The Nordic Semiconductor development interface, nRF Connect, already has a library to include a Texas Instruments TLC59108, a device similar to the one used in this project. In the hopes that it was similar enough, it was added to the devietree using the graphical user interface in nRF Connect as shown in fig. 3. All this library does is include the TLC as a generic I²C-device, which is what would have been done anyways.

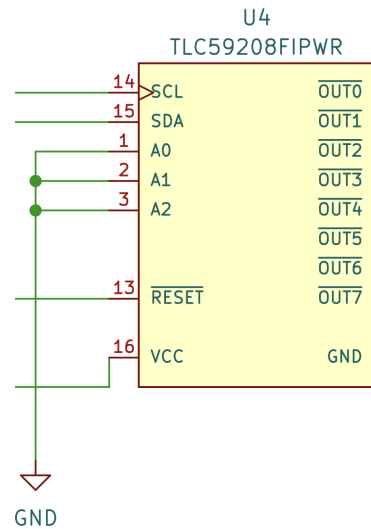


Figure 2: Address inputs on the TLC [2].

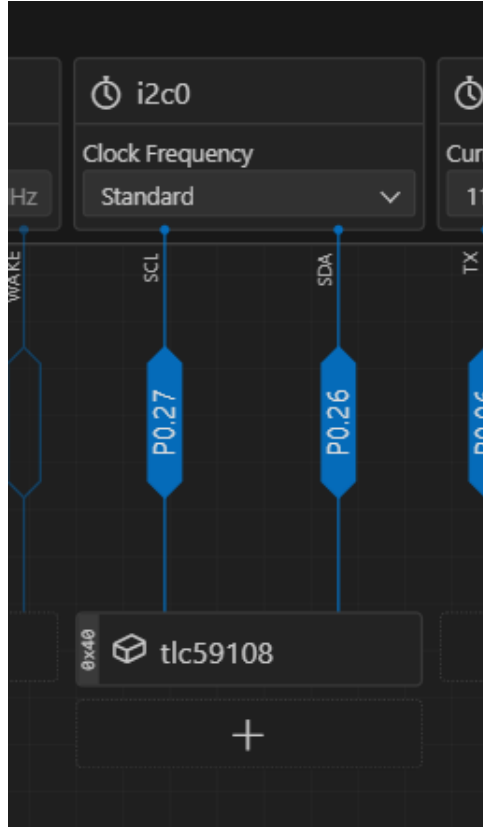


Figure 3: Adding TLC to devicetree using the graphical user interface.

1.1.3 Registers

When the devicetree and I²C is ready, the control registers can be written to. This project uses the default settings of the TLC for all values except one. Upon power up, the default setting for the TLC is to enter *low power mode*. In this application there is always one LED on, and there is no need to save power, therefore the device is instead configured to be in normal mode.

1.2 Control

Once the configuration is done, the device is ready to be used. In this project, the TLC used to loop through six LED, turning them on and off one at a time. Figure 4 shows how the registers should be written to (for the second of the 2 LEDOUT registers, but it is the same for the first). The 8 bits in the register are divided into four groups of 2, where each group represents one LED. To turn a LED fully on, the value 0b01 must be written to the appropriate group.

1.3 Testing

Figure 5 shows the flow of the loop implemented to test the TLC, after the configuration is done. To confirm that the implementation has been successful, it is sufficient to see that the LEDs go on and

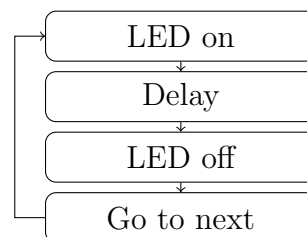


Figure 5: Test cycle.

ADDRESS	REGISTER	BIT	SYMBOL	ACCESS ⁽¹⁾	VALUE	DESCRIPTION
0Dh	LEDOUT1	7:6	LDR7[1:0]	R/W	00 ⁽²⁾	LED7 output state control
		5:4	LDR6[1:0]		00 ⁽²⁾	LED6 output state control
		3:2	LDR4[1:0]		00 ⁽²⁾	LED5 output state control
		1:0	LDR4[1:0]		00 ⁽²⁾	LED4 output state control

LDRx = 00 : LED driver x is off (default power-up state).
LDRx = 01 : LED driver x is fully on (individual brightness and group dimming/blinking not controlled).
LDRx = 10 : LED driver x is individual brightness can be controlled through its PWMx register.
LDRx = 11 : LED driver x is individual brightness and group dimming/blinking can be controlled through its PWMx register and the GRPPWM registers.

Figure 4: LEDOUT1 register [1].

off in a circle. However, the LEDs emit wavelengths between 940nm and 1550nm, which are invisible to the human eye. In order to confirm that the LEDs actually turn on, they can be observed by using a camera that doesn't filter these frequencies. Alternatively, voltages across the LEDs can be measured. This will not confirm that they light up, only that they get voltage.

2 Discussion

In this section we will discuss an issue with ambiguity of the I²C-address for the TLC, point out some things the Arduino-code does inefficiently, and address an issue with LED visibility encountered during testing.

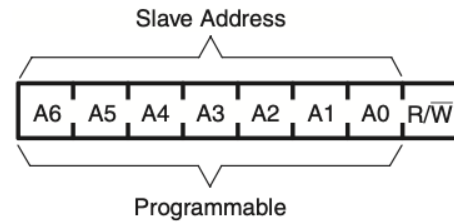


Figure 6: Adress scheme of the TLC [1].

2.1 I²C-address

Some time was spent troubleshooting due to having the wrong I²C-address. Figure 1 shows the address to be 0x20, but to the right of this figure it says "Address: 40h". This was interpreted to mean that the address should be 0x40, which is wrong. The ambiguity is with wether or not the R/W-bit shown in fig. 6 is part of the address or not. This bit is part of the *register which holds the address*, but is not a part of the address itself. This means that when the register has the value 0x40, the address is 0x20.

2.2 Control registers

When the TLC is configured for this application, only one register value is changed, and the default settings are used for the rest. The Arduino code does this differently to this project. It utilizes the *auto-increment* functionality of the TLC, which lets it receive an number of values in a row, and the values are sent to register addresses in ascending order [1]. Using this, nearly *all* registers are

```
Wire.beginTransmission(ADDR);
Wire.write(0x80); // Control
Wire.write(0x81); // 00h: M
Wire.write(0x00); // 01h: M
Wire.write(0x00); // 02h: P
Wire.write(0x00); // 03h: P
Wire.write(0x00); // 04h: P
Wire.write(0x00); // 05h: P
Wire.write(0x00); // 06h: P
Wire.write(0x00); // 07h: P
Wire.write(0x00); // 08h: P
Wire.write(0x00); // 09h: P
Wire.write(0x00); // 0Ah: G
Wire.write(0x00); // 0Bh: G
Wire.write(0xAA); // 0Ch: L
Wire.write(0xAA); // 0Dh: L
Wire.write(0x00); // 0Eh: S
Wire.write(0x00); // 0Fh: S
Wire.write(0x00); // 10h: S
Wire.write(0x00); // 11h: A
```

Figure 7: Register values in the Arduino code [3].

written to, as shown in fig. 7 [3]. Most of the values written here are 0x00, which is mostly default values. A cleaner solution would be to only write to the registers that need configuring. This makes the setup faster, and the code more readable.

2.3 PWM vs. LEDOUT

Another difference between the code for this project and the Arduino code, is that the Arduino code uses pulse width modulation (PWM) to control the LEDs. The documentation for the TLC states that the PWM operates at 97 kHz, with a maximum duty cycle of 99,6% [1]. The purpose of PWM in the TLC is to dim the lights for applications where full brightness is undesired. In this application there is no reason to use the LEDs below full brightness. Using PWM also gives a small (0.4%) chance that a LED will be dark when a scan is being done, because of the 99,6% duty cycle. In this project, the LEDOUT register of the TLC is written to directly, causing LEDs to either be fully on or fully off.

2.4 Visibility of LEDs

As stated previously, the LEDs which are being controlled emit wavelengths invisible to the human eye, but they can be seen by using a camera that doesn't filter these frequencies. It should be noted that some cameras have more effective filters than others. The phone cameras which were tested all picked up D4 and D5 (940 and 1050nm). D6 and D7 (1200 and 1300nm) was detectable with some cameras and the light from LEDs D8 and D9 (1460 and 1550nm) has not been detected by any of the cameras that have been tested. The code has been confirmed as working by instead measuring the voltage across these LEDs.

3 Conclusion

This report has discussed the implementation of a TLC59208F LED driver as part of the plastic scanner. The necessary steps for successful implementation has been explained, and some improvements from the Arduino code being mimicked [3] have been pointed to. Namely, writing to the LEDOUT registers rather than using PWM for better control of the LEDs, as well as not writing default values to registers, for a more effective setup. An ambiguity in the I²C-address, where the value of the address is different from the value contained in the register which holds the address, has also been clarified. Lastly, a challenge with LEDs emitting invisible wavelengths has been addressed.

References

- [1] Texas Instruments, “TLC59208F,” 2015, last accessed 13 May 2025. [Online]. Available: <https://www.ti.com/product/TLC59208F/part-details/TLC59208FIPWR>
- [2] J. de Vos, “DB 2.3,” 2023, last accessed 13 May 2025. [Online]. Available: <https://docs.plasticscanner.com/boards/DB2.3>
- [3] —, “DB2.x Firmware,” 2022, last accessed 13 May 2025. [Online]. Available: <https://github.com/Plastic-Scanner/DB2.x-Firmware>